



UNIVERSIDAD DE EXTREMADURA

FACULTAD DE CIENCIAS

Grado en FÍSICA

**MEMORIAS DE PRÁCTICAS
EXTERNAS**

JAVIER MOLINA PÉREZ
JULIO, 2025

Índice:

1. Introducción y antecedentes generales.....	2
1.1 ¿Qué es FIWARE Space?.....	2
2. Estado del arte.....	3
3. Resumen.....	4
4. Memoria descriptiva.....	4
5. Proyectos.....	15
5.1. Postman-Aeropuerto.....	15
5.2. Python-Primeros pasos.....	16
5.3. Respira.....	18
5.4. Sensorización aeropuerto y conectividad con Postman.	19
6. Conclusiones.....	24
7. Bibliografía.....	25

1. Introducción y antecedentes generales.

Este documento es el informe final en el que se recogen en que han consistido, cuales eran los objetivos así como las conclusiones a las que se han llegado después de haber terminado las prácticas curriculares (asignatura optativa del grado en física).

El periodo de prácticas se ha extendido desde el 3 de junio de 2025 hasta el 11 de julio de ese mismo año, entrando dentro del curso académico 2024-2025. Cabe notar que este periodo de prácticas no ha sido continuo ya que se dividió en dos partes (del 3 al 16 de junio y del 3 al 11 de julio) con objeto de permitir la preparación de los exámenes de la convocatoria extraordinaria así como porque en el mes de julio los tutores nos informaron de la dificultad de realizarlas, debido a periodos de vacaciones.

Las prácticas no las he realizado en solitario sino que las realicé con una compañera del grado, Sarah González Abadito.

Los tutores en la empresa fueron Miguel López Corbacho y Jorge Osuna Pons.

1. 1 ¿Qué es FIWARE Space?



Figura 1: Logo de el centro y la diputación

FIWARE Space recibe su nombre de la tecnología Fiware. Esta tecnología surge de una iniciativa de la Unión Europea con el objetivo de proporcionar un estándar a nivel europeo a la hora de gestión de datos e internet de las cosas (al que a partir de este momento nos referiremos como IoT).

El centro FIWARE Space nace en el seno (y como una de las puntas de lanza) de una iniciativa de la diputación de Badajoz, “Badajoz Es Más”.

Esta iniciativa responde a un creciente interés en modernizar la infraestructura tecnológica de la región gracias a los nuevos y cada vez más veloces avances en este campo. Su actividad se centra en 6 campos principales:

- 1: *Urban*. Siguiendo la filosofía de el IoT y las “smart cities” (ciudades inteligentes), sensorización de distintos elementos que hay en las ciudades y con ello acceder a los datos para realizar estudios.

- 2: *Center*. Impulsar el centro (que ya constituye un FIWARE i-Hub, lugar de referencia a nivel internacional para la formación y certificación de productos basados en FIWARE) como núcleo de dinamización de proyectos y emprendimiento.
- 3: *Platform*. Construir una plataforma para gestionar de forma inteligente y horizontal los servicios públicos así como ofrece soluciones basadas en IoT. Asimismo los datos recabados estarán en la plataforma de forma abierta para su estudio.
- 4: *University*. El centro tiene una serie de convenios con la Universidad de Extremadura con el objeto de realizar distintas actividades que fomenten la investigación, el desarrollo y la innovación.
- 5: *Challenge*. Realizar cada año eventos y actividades (con dotación económica proveniente de la diputación) para encontrar soluciones a problemas y necesidades reales basándose en el FIWARE.
- 6: *Market*. Se trata de conseguir que las soluciones usadas por empresas y basadas en FIWARE sean conocidas a lo largo del mundo.

2. Estado del arte.

En esta sección del informe el objetivo es exponer la situación actual y el punto de desarrollo en el que se encuentran las tecnologías usadas.

Como ya habíamos establecido el centro se basa principalmente en la aplicación de tecnología FIWARE así que hablemos primero de ella.

Para poder hacerlo más sencillo veamos un esquemático de como se estructura la misma:

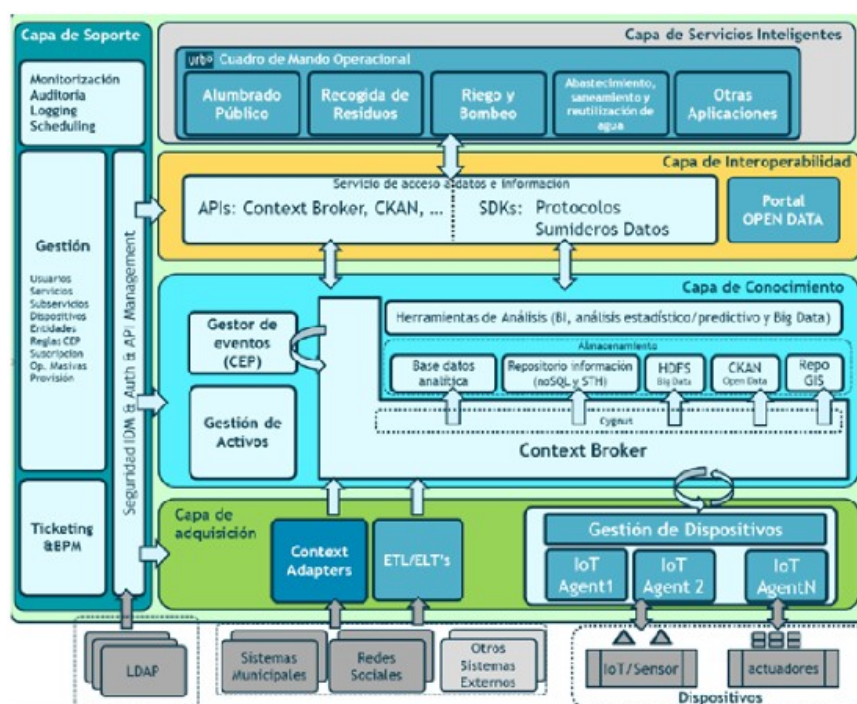


Figura 2: Arquitectura de la tecnología FIWARE

Si nos fijamos en la *figura 2* podemos observar que la estructura general de esta tecnología esta dividida en un modelo de capas, dentro de las cuales hay diversos elementos. Por orden de acción tenemos:

- Los sensores, actuadores etc, véase los elementos que suministrarán los datos a estudiar y tratar.
 - La capa de soporte. Esta capa posee herramientas para la gestión de datos, usuarios, logging, etc. Es una capa cuyo objetivo es poder filtrar la información recibida.
 - La capa de adquisición. En esta se encuentran los componentes, herramientas y funcionalidades que permiten recibir (o, como su propio nombre indica adquirir) los datos suministrados por los sensores.
 - La capa de conocimiento. Esta capa es esencial pues en ella se encuentra el context broker que es si no lo esencial de el FIWARE uno de sus elementos esenciales. Este tiene los métodos y las herramientas para recibir los datos que salen de la capa de adquisición y gestionarlos véase almacenarlos, recopilarlos y distribuirlos en función de su contexto.
- En esta capa hay herramientas relacionadas con el mundo de el análisis de datos con el objetivo de poder analizar los datos ya recopilados por el context broker.
- La capa de interoperabilidad. Esta capa se encarga de facilitar la conexión y comunicación con otros sistemas y aplicaciones a través de, por ejemplo, API Keys.
 - La capa de servicios inteligentes. Esta capa contiene los cuadros de mando ejecutivo y operacional. Tienen diferentes recursos o funcionalidades.

3. Resumen.

Durante el periodo de prácticas se ha realizado trabajo y estudio de diversos tipos de tecnología y programas de importancia capital en el ámbito de la sensorización, IoT y Fiware. Con objeto de salir con nuevas herramientas útiles en nuestra carrera profesional.

4. Memoria descriptiva.

En esta sección se verá por encima lo que se vio a lo largo del periodo de prácticas para en el siguiente apartado desglosar y desarrollar de forma más extensa aquellas áreas en las que más hincapié se haya hecho porque hayan estado involucradas en los proyectos algo más ambiciosos que han sido realizados.

·Orange data mining:



Figura 3: Logotipo de Orange Data Mining.

Esta fue una de las primeras tareas que se nos propuso al llegar después de una introducción. Yo trabajé un poco con este programa mientras que mi compañera dedicó su tiempo a estudiar otro de una naturaleza similar, Knime.

Ambos programas tienen como objetivo simplificar el análisis de datos o, más bien, el simplificar la construcción de funciones para ello. Esto lo hacen haciendo uso de una programación en bloques que hacen muy sencillo su aprendizaje. Aunque con todas las partes y los campos en los que nos introducimos no hubo mucho tiempo de profundizar en este gracias a los tutoriales que ofrece la propia aplicación todavía se pudo hacer algún programa interesante.

·Un analizador de imágenes de animales. Este programa lo que hace es recibir un grupo de imágenes de animales que le suministramos nosotros y aunque la clasificación ya se la damos hecha en grupos más grandes (reptil, anfibio, mamífero, etc.) es capaz de detectar aquellos que sean más similares entre sí.

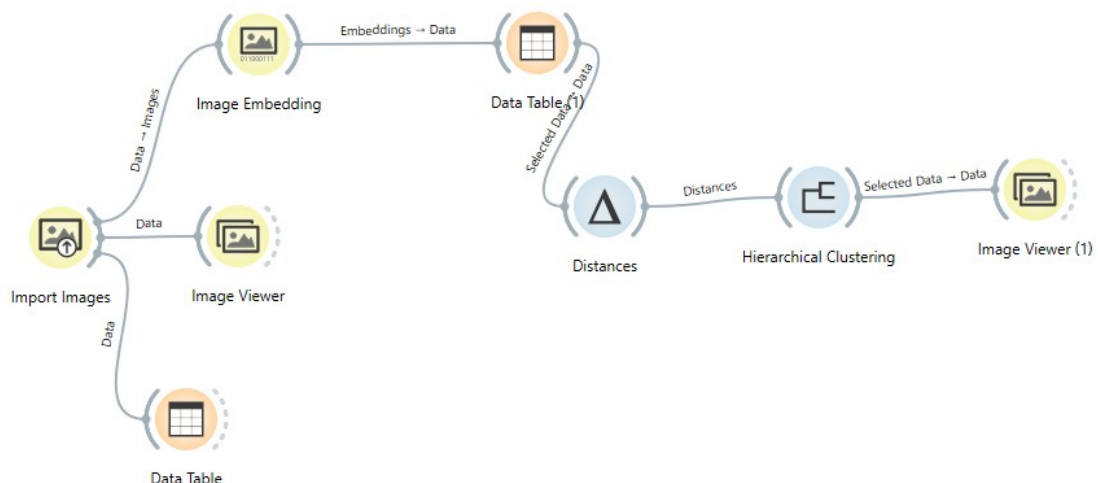


Figura 4: Imagen del código que realiza la función

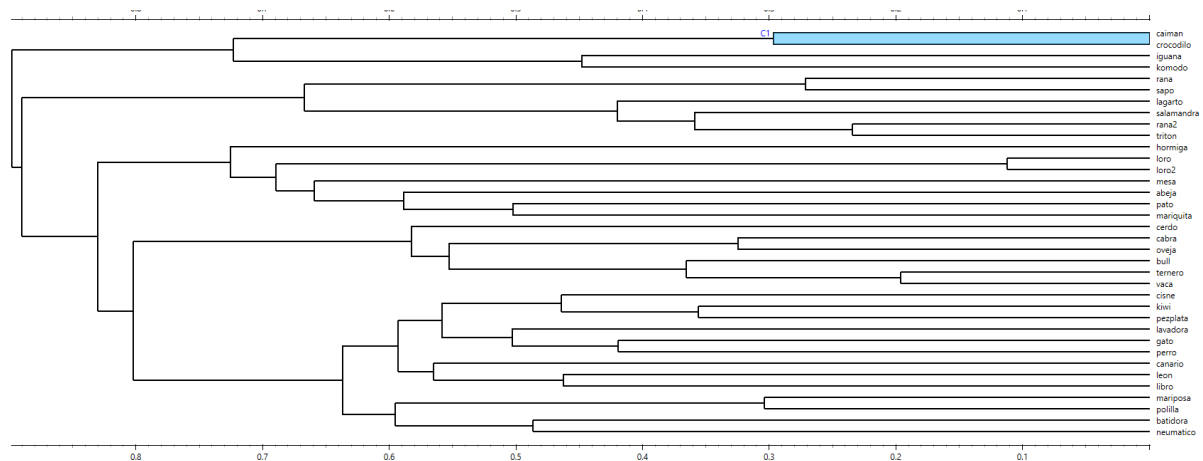


Figura 5: Representación de como clasifica las imágenes.

Otra cosa que era capaz de hacer este programa era que si se le introducía una nueva imagen, pongamos un caballo (es decir un mamífero) que no estaba dentro de el grupo de imágenes de referencia el propio programa ya es capaz de colocarlo en la propia categoría de mamífero. Esto lo hace ya que lo que está haciendo el programa es con la función de *Distances* que se ve en la *figura 4* desglosa la imagen en píxeles y va encontrando similitudes entre estas entonces cuando encuentra suficientes similitudes con las que tiene como referencia de Mamífero la meterá ahí.

Basándonos en este mismo grupo de imágenes hicimos otro programa que lo que hacía era algo similar ya que volvía a clasificar las imágenes pero en este caso trataba de clasificar las nuevas imágenes en las anteriores categorías, dando porcentajes de error así como representando los resultados en lo que se conoce como una *matriz de confusión*.

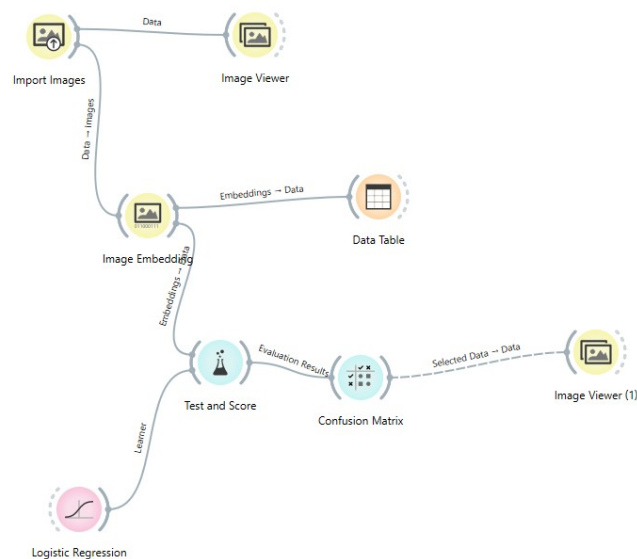


Figura 6: Programa de clasificación de animales.

		Predicted						
		Anfibio	Ave	Insectos	Mamífero	No Animal	Reptil	Σ
Actual	Anfibio	5	0	0	0	0	0	5
	Ave	0	3	2	1	0	0	6
	Insectos	0	2	4	0	0	0	6
	Mamífero	0	0	0	9	0	0	9
	No Animal	0	1	0	3	1	0	5
	Reptil	1	0	0	0	0	4	5
Σ		6	6	6	13	1	4	36

Figura 7: Ejemplo de la mentada matriz de confusión.

Un último pequeño programa que hicimos fue uno de análisis de textos que tenía dos funciones principales.

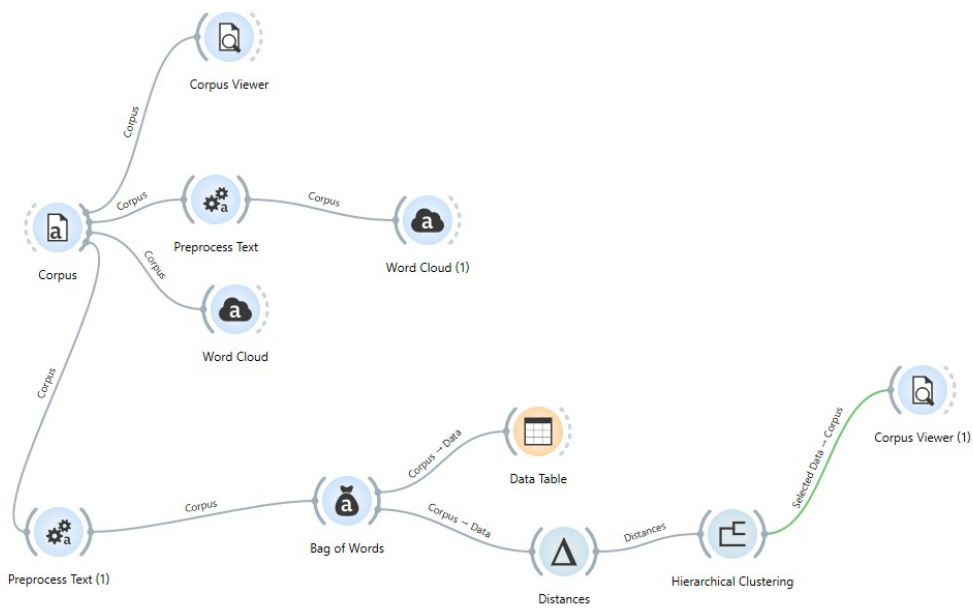


Figura 8: Programa de análisis de textos.

Su primera función es la que realiza la rama de arriba, esta analiza el texto, elimina los conectores (al ser inglés palabras como *the*, *and*, *etc*) para luego recibir una nube de palabras en las cuales a medida que la palabra va tomando un lugar central y se va haciendo más grande su frecuencia es mayor en el texto (útil quizás para poder ver de un vistazo rápido de qué trata el texto analizado).

·MQTT/Mosquitto:



Figura 11: Logotipo de Mosquitto.

Otra tecnología con la que también probamos un poco fue el MQTT, en concreto a través de la aplicación de Mosquitto. El que nos explicó sobre el uso de esta aplicación fue Jorge Osuna.

MQTT hace referencia a un sistema de comunicación entre servidor y dispositivos. Se caracteriza por ser muy ligero y por esto requerir de una potencia de computación muy reducida.

Es el estándar para la comunicación en IoT (que como ya dijimos es uno de los focos del centro, de ahí que nos lo explicaran). También destaca por ser un sistema de comunicación relativamente seguro gracias a su protocolo criptográfico TLS (*Transport Layer Security* o Seguridad de la capa de transporte, que es un estándar para la encriptación de comunicación entre servidores y aplicaciones web).

Mosquitto (más concretamente Eclipse Mosquitto) es una aplicación utilizada a tal efecto, sirve para poder conectar elementos y enviar mensajes a través de MQTT.

Asimismo cabe destacar que se trata de una aplicación de código abierto, véase que una vez descargas el software esa versión descargada pasa a ser tuya y ya no te podría ser retirado su uso, como si podría pasar con sistemas de suscripción, por ejemplo. (En el apartado **Conclusiones** hablaré un poco sobre ello ya que me parece algo destacable del centro).

Esta tecnología al final lo acabamos usando un poco de forma anecdótica, por desgracia, ya que no hubo tiempo de mucho más. El día que la vimos probamos a mandar unos mensajes tanto mi compañera y yo desde nuestros ordenadores personales a uno que tienen en abierto en el centro, con el objetivo de hacer pruebas.

Hubiera estado bien poder haber incluido alguna clase de conectividad a través de MQTT en el proyecto pero estos conocimientos básicos plantan una semilla que podrá germinar si acabamos ahondando más en la misma.

·Postman:



Figura 12: Logotipo de Postman.

Esta fue la siguiente aplicación más destacable que utilizamos a lo largo de este periodo, Miguel se encargó de introducirnos a su uso.

Esta aplicación simplifica mucho el proceso de trabajar con las anteriormente mencionadas API Keys. En el propio entorno de trabajo que ofrece ya está todo aquello se podría necesitar para trabajar con estas.

Todo gira entorno a generar estas API Keys por nosotros mismos o testear las generadas por otras personas.

También es sumamente útil para gestionar desde el mismo programa todos los datos y los sensores que proporcionan los mismos gracias a su lógica interna.

En Postman se crean lo que se conocen como entidades, las cuales harán referencia a lo que sea que queramos estudiar, por ejemplo, un sensor de movimiento). Asimismo las entidades pueden seguir una jerarquía y anidar unas dentro de otras, véase una entidad puede ser Habitación y dentro de esa entidad hay numerosas entidades, sensor de movimiento, de luz, etc.

También cabe destacar que las entidades aparte de estar unas anidadas dentro de otras pueden tener diversas cualidades que toman valores como valor enteros, booleanos, etc. con el objetivo de hacerlas lo más flexibles posible.

En Postman hay 3 tipos de llamada principales: Get, Post y Del.

-Get: Es una llamada que nos permite ver lo que sea que deseemos ver (si necesitamos ver una entidad en concreto usamos la llamada *Get a single entity* y podremos verla).

-Post: La llamada que permite crear o modificar entidades.

-Del: El último tipo de llamada general que hay. Es la que permite eliminar entidades o atributos.

En el siguiente apartado **4. Proyectos** volveremos a Postman y veremos para que lo usamos, ya que Miguel nos propuso un proyecto para usarlo así como luego lo utilizaríamos para el proyecto grande que sirve como proyecto final de las prácticas.

·SQL:



Figura 13: Logotipo de SQL.

A lo largo de nuestra estancia de prácticas también dedicamos tiempo a probar diversos lenguajes de programación. Uno de estos fue SQL.

SQL es un lenguaje que se utiliza de forma común para gestionar bases de datos y tablas. Es un lenguaje sencillo e intuitivo pues su objetivo era ser lo más fácil de usar para que así cualquier profesional, ya tenga una base de informática o no pueda entenderlo y usarlo.

Este lenguaje, así como el siguiente no nos acabó de dar tiempo a exprimirlo demasiado debido a lo que ya se ha mencionado de que íbamos pasando de una cosa a otra, con el objetivo de tener un abanico más grande de posibilidades, para después ahondar en aquello que más nos llamara la atención.

El aprendizaje de este lenguaje se basó principalmente la realización de una serie de tutoriales en la página web *W3 Schools*.

El lenguaje en pos de ser lo más sencillo posible tiene algunas funciones generales bastante autoexplicativas:

-Select: Selecciona una determinada base de datos o tabla, así como determinados datos dentro de las mismas.

-Insert: Permite añadir nuevos datos a base de datos o tabla.

-Update: Su función es actualizar los valores de los datos ya existentes.

-Delete: Elimina los datos o tablas seleccionados.

Existen más funciones pero en líneas generales estas son la base y a partir de ahí se construyen los programas con SQL.

Este lenguaje puede ser usado con aplicaciones como MySQL para realizar el *backend** de una página web.

De hecho sobre hacer páginas web se habló de ver un poco basándonos en la aplicación Wamp y con la ayuda del MySQL pero al final no hubo tiempo.

**Backend* Hace referencia a la parte de una página web que el usuario promedio no ve, es aquella que realiza toda la lógica y la gestión de datos. La parte “bonita” de la web, con el

texto, las imágenes y en general todo lo relacionado con hacer la experiencia de navegarla lo más cómoda posible para el usuario se denomina como *frontend* y uno de los lenguajes más usados para hacerlo será HTML.

•**Javascript:**



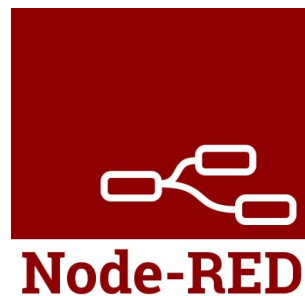
*Figura 14:
Logotipo de
Javascript.*

Otro lenguaje que empezamos a ver un poco fue Javascript. La página usada a tal efecto fue otra vez la anteriormente mencionada *W3 Schools*.

Este lenguaje se utiliza también para el *frontend* de las páginas web. Mientras que HTML lidia con la estructuración de la página y la información que se muestra en pantalla este trata de añadir la capacidad de interactuar con la misma. (Como apunte el otro gran lenguaje que se usa para la gestión de esta parte de las páginas web sería CSS pero con ese sí que no lidiamos en ningún momento).

Por desgracia para con este lenguaje no dio demasiado tiempo antes de pasar a otras cosas.

•**Node-red:**



*Figura 15: Logotipo
de Node-red.*

Siempre hay formas más sencillas de realizar una misma tarea. Una forma de simplificar el hecho de realizar aplicaciones gracias a su estructura de código en bloques es Node-red.

Su función es la de que cualquiera pueda realizar aplicaciones gracias a las cuales visualizar, gestionar o transformar los datos deseados.

Los códigos siguen una estructura de flujo que recuerda mucho a un esquema, lo que lo hace todo muy visual.

Cada bloque tiene una función muy concreta y precisa que, con las librerías que se pueden descargar y son de dominio público permiten hacer toda clase de programas.

Aunque este programa tampoco lo llegamos a usar tanto como nos hubiera gustado aún nos dio tiempo de hacer algún programa curioso y que deja constancia de las posibilidades que ofrece esta aplicación.

•**Arduino:**



Figura 16: Logotipo de Arduino.

Probablemente esta es la tecnología a la que más tiempo le dedicamos y es de hecho entorno a la que giró ese proyecto algo más grande que ya ha sido mencionado.

Arduino se refiere a una empresa que manufactura placas de desarrollo de hardware gracias a los microprocesadores que van incluidos en las mismas. Todo es de código abierto y tiene una gran comunidad que se dedica a crear programas y publicarlos así como hay numerosos foros en los que se ofrece ayuda.

El lenguaje de programación usado para comunicarse con el microprocesador es C.

Arduino, al ser de libre es muy maleable, sobre todo debido a que al tener tanta demanda hay toda clase de conexiones y extensiones que se le pueden añadir, en forma de librerías. Asimismo también hay una inabarcable colección de sensores, emisores, etc. que permiten la conectividad con las placas Arduino, con lo que se puede realizar toda clase de circuitos con usos diversos.

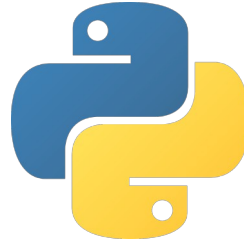
Y eso no es todo, en el centro lo que había era la placa Arduino UNO R4, que permite la conectividad con redes wifi para emitir y recibir señales a través de esta.



Figura 17: Placa Arduino UNO R4.

Las placas Arduino permiten la entrada y salida de datos analógicos y digitales. A través de código se especificará que puertos conectados emitirán y cuales recibirán información, así como la función de cada uno.

·**Python:**



*Figura 18:
Logotipo de
Python.*

Lo último que vimos en el centro fue el lenguaje de programación Python. El uso de este lenguaje nos fue introducido en nuestros últimos días de la mano de Miguel López.

Python es un lenguaje sencillo y potente, lo que permite que sea usado a lo largo de todo el mundo y de hecho lo convierte en un lenguaje idóneo para aprender. También permite que aquellas personas con un trasfondo que necesite usar código pero quizás no tengan una gran habilidad para ello puedan comenzar a hacer unos primeros programas que les ayuden a su tarea de forma intuitiva.

Es cierto que debido a su estructura y su funcionamiento (lee el código línea a línea) es un lenguaje más lento, pero eso no nos supuso un problema para los primeros programas que hemos hecho. Eso se puede volver un problema en códigos muy extensos o complejos.

En el apartado de **Proyectos** veremos algunos de los usos que le hemos dado.

5. Proyectos.

5.1 Postman-Aeropuerto.

En este apartado de proyectos hablaré de los tres que me parecen más relevantes o que siento que acabarán resultando algo más útiles.

El primero vino después de ver un poco las bases de Postman de la mano de Miguel. Nos propuso el crear una entidad algo más grande para anidar diversas entidades dentro de la misma y con ello comprender el proceso de crear y gestionar entidades en la aplicación.

Después de deliberar un poco Sarah y yo decidimos hacer una entidad Aeropuerto dentro de la cual meteríamos las demás. Creamos entidades que introducir en esta más general como terminales, pistas de aterrizaje o incluso vuelos.

```
{
  "id": "aeropuerto001",
  "type": "aeropuerto",
  "abierto": {
    "type": "Boolean",
    "value": false,
    "metadata": {}
  },
  "numeroDePersonas": {
    "type": "Number",
    "value": 0,
    "metadata": {}
  }
},
```

Figura 19: Entidad aeropuerto.

```
{
  "id": "terminal001",
  "type": "terminal",
  "abierto": {
    "type": "Boolean",
    "value": false,
    "metadata": {}
  },
  "numeroDePersonas": {
    "type": "Number",
    "value": 0,
    "metadata": {}
  },
  "refAeropuerto": {
    "type": "Relationship",
    "value": "aeropuerto001",
    "metadata": {}
  }
},
```

Figura 20: Entidad Terminal001

```
"id": "vuelo001",
"type": "vuelo",
"aerolinea": {
  "type": "String",
  "value": "Ryanair",
  "metadata": {}
},
"avion": {
  "type": "String",
  "value": "Boeing-747",
  "metadata": {}
},
"destino": {
  "type": "String",
  "value": "Roma-Fiumicino",
  "metadata": {}
},
"horasalida": {
  "type": "String",
  "value": "18:00",
  "metadata": {}
},
"horallegada": {
  "type": "String",
  "value": "20:40",
  "metadata": {}
},
```

Figura 21a: Entidad vuelo001

```
"horaSalida": {
  "type": "String",
  "value": "18:00",
  "metadata": {}
},
"numeroDePasajeros": {
  "type": "Number",
  "value": 0,
  "metadata": {}
},
"puertaEmbarque": {
  "type": "String",
  "value": "1",
  "metadata": {}
},
"refTerminal": {
  "type": "Relationship",
  "value": "pistaAterrizaje001",
  "metadata": {}
},
```

Figura 21b: Entidad vuelo001

En los ejemplos que se incluyen se puede ver que el uso de la aplicación es bastante sencillo y con las llamadas de *Create a single entity* y *Update a single entity* al final acabábamos haciendo casi todo.

Este proyecto del aeropuerto serviría como idea y base del proyecto final, que era sensorizar (de forma muy simple) un aeropuerto, los carriles de la pista de aterrizaje, temperatura y niebla, así como añadir conectividad con Postman y que dicha información se fuera recogiendo y subiendo, para poder acceder a ella de forma remota.

5.2 Python-Primeros pasos.

En esta sección hablaremos un poco de los primeros programas que nos dio tiempo a hacer con a Python. Probablemente sea la sección más breve en esta parte de proyectos (ya que como se mencionó anteriormente, solo dedicamos los últimos días a esto) pero considero oportuno añadirla aquí con la importancia que merece ya que Python es un lenguaje sumamente utilizado en el ámbito profesional y probablemente estas bases que hemos adquirido nos sean útiles para desarrollarlo más.

Debido a que Python trabaja mucho con listas la mayor parte del trabajo giró entorno a estas.

Como primer programa algo más elaborado (por supuesto empezamos por el clásico “Hello World” y similares) Miguel nos propuso hacer un programa que con una lista de números del 0 al 5 se creara otra lista cuyos elementos fueran la suma de todos los anteriores de la primera (esto es, el tercer elemento de la segunda lista será la suma de los tres primeros elementos de la primera, por ejemplo). El resultado fue el siguiente:

```
17 lista1=[0, 1, 2, 3, 4, 5]
18 lista2=[]
19 #for i in lista1:
20 | # lista2.append(i+i)
21 #print(lista2)
22 #for k in range(5):
23 | # print(k)
24 cont=0
25 suma=0
26 for i in lista1:
27 | suma=0
28 | print(f"i + {i}")
29 | #print(cont)
30 | for k in range(cont+1):
31 | | print(f"k + {k}")
32 | | suma+=lista1[k]
33 | | print(suma)
34 | lista2.append(suma)
35 | cont+=1
36 print(lista2)
```

PROBLEMS OUTPUT DEBUG CONSOLE

[0, 1, 3, 6, 10, 15]

Figura 22: Primer código de Python.

Para el siguiente volvíamos a trabajar con listas pero en este caso el objetivo era contar el número de palabras que había en un párrafo así como detectar la posición de una palabra que haya en el propio párrafo, introducida por el usuario. El resultado fue el siguiente:

```

2 texto=input("Introduce un párrafo: ")
3 busq=input("Introduce una palabra y veremos si existe en el texto así como su posición en el mismo: ")
4 esp=texto.count(" ") + 1
5 print(f"El número de palabras es: {esp}")
6 #pal=texto.find(busq)
7 div=texto.split()
8 cont=0
9 for i in div:
10     cont+=1
11     if i==busq:
12         print(f"La palabra {busq} se haya en la posición {cont}")
13
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
1
PS C:\Users\Javi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Javi\AppData\Local\Programs\Python\Python38-32\python.exe c:/Users/Javi/D
Introduce un párrafo: Te alegrara saber que no ha acompañado ninguna desgracia al comienzo de una empresa que tú veías con tan malos augurios.
Introduce una palabra y veremos si existe en el texto así como su posición en el mismo: desgracia
El número de palabras es: 21
La palabra desgracia se haya en la posición 9

```

Figura 23: Segundo código de Python.

El tercer y último código más interesante que vimos estuvo relacionado con comenzar a introducirnos a trabajar con diccionarios, también de una importancia imperial a la hora de lidiar con Python. Estos funcionan de una forma similar a las listas en tanto a que guardan numerosos valores pero al contrario que en las listas en los diccionarios no se guardan de forma ordenada sino que los datos se guardan como un pareja de clase y valor ["clase":valor]. En este último ejercicio lo que se nos pedía era un código en el que al introducirle un texto este guarde en un diccionario todas las palabras de tal manera que la clase es la palabra y el valor sea el número de veces que se repite la misma. El resultado fue el siguiente:

```

6 texto=input("Introduce el texto a convertir ")
7 print(texto)
8 t_list=texto.split()
9 dic={}
10 suma=0
11 suma2=0
12 suma3=0
13 ind=[]
14 dist=[]
15 indi=0
16 for i in t_list:
17     suma=0
18     dist.append(i)
19     for j in t_list:
20         if i==j:
21             suma+=1
22     dic[i]=suma
23 #print(f"El número de veces que se repite {i} es: {suma}")
24 print(dic)
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Javi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Javi\AppData\Local\Programs\Python\Python38-32\python.exe c:/Users/Javi/Desktop/Python/diccionarios.py
PS C:\Users\Javi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Javi\AppData\Local\Programs\Python\Python38-32\python.exe c:/Users/Javi/Desktop/Python/diccionarios.py
Introduce el texto a convertir Esta yegua no es mi vieja yegua gris.
Esta yegua no es mi vieja yegua gris.
{'Esta': 1, 'yegua': 2, 'no': 1, 'es': 1, 'mi': 1, 'vieja': 1, 'gris.': 1}

```

Figura 24: Tercer código de Python.

También trabajamos un poco con librerías (*fpdf* y *requests*) aunque fue ya al final y acabo siendo algo un poco anecdótico.

5.3 Respira.

El lunes 16 de junio hicimos algo distinto. Nos ofrecieron ir a pasar la jornada a Olivenza, a visitar su CitLAB y dedicar la mañana a ayudar en el proyecto Respira.

El CitLAB forma parte de el mismo afán de modernizar la región que dio lugar a FIWARE Space. Es un centro (como otros similares hay en Jerez de los Caballeros y Villanueva de la Serena) en el que hay todo lo necesario para poder diseñar y construir dispositivos electrónicos y de IoT para buscar nuevas soluciones. Asimismo esta abierto al público para que aprenda quien lo desee y un buen ejemplo de ello es el proyecto Respira.

Respira es una iniciativa la cual busca establecer una red de dispositivos IoT, conectados gracias a la tecnología Fiware en la cual el ciudadano se vea involucrado. Se montan talleres en sitios como los CitLAB en los cuales cualquiera puede ir y se les suministrará los materiales necesarios (sensores, microprocesador y estructura impresa en 3D) para montar cada uno su propia estación que mide gases, partículas, temperatura y humedad. Luego la cosa es que cada uno lo coloque en un punto distinto para así recabar información de diversos puntos a lo largo de la región.

Estos datos, continuando con la misma filosofía estarán en abierto y se podrán mirar en cualquier momento desde la página web del proyecto.

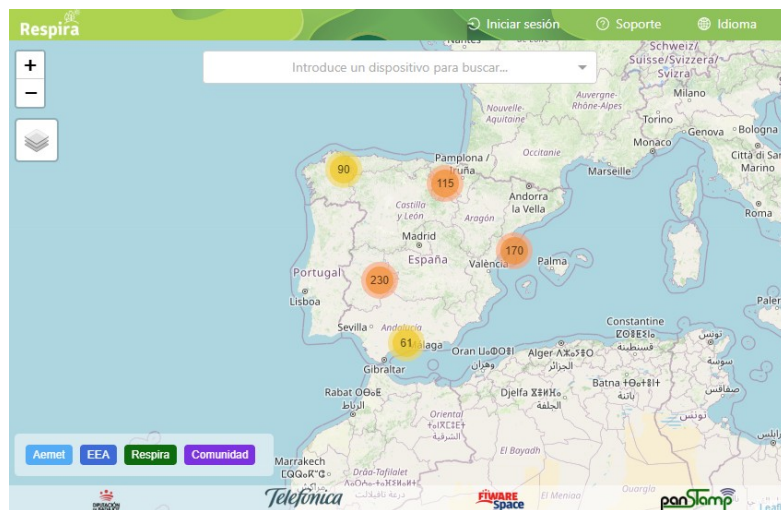


Figura 25: Ejemplo de como se ve el mapa de Respira.

PM10	2.77 $\mu\text{g}/\text{m}^3$	13/6/2025 14:54
PM10	2.62 $\mu\text{g}/\text{m}^3$	13/6/2025 14:54
PM2.5	2.77 $\mu\text{g}/\text{m}^3$	13/6/2025 14:54
PM4.0	2.77 $\mu\text{g}/\text{m}^3$	13/6/2025 14:54
Humedad relativa	41.6 %	13/6/2025 14:54
no2	46.19	13/6/2025 14:54
Temperatura	30.7 °C	13/6/2025 14:54
Tamaño típico de partícula	0.47 μm	13/6/2025 14:54
AQI	3	13/6/2025 14:54

Figura 26: Datos tomados por una unidad Respira.

Nuestro papel fue asistir junto a Miguel y Jorge para realizar el proceso de montar una estación Respira y ver como de bien se pueden seguir las instrucciones, para que cuando lo abran al resto del mundo la experiencia la pueda seguir quien sea, independientemente de su nivel de formación. La experiencia fue satisfactoria y, aunque había alguna pequeña arista que pulir para ser una de las primeras veces que se realizaba esto en este centro fue muy bien.

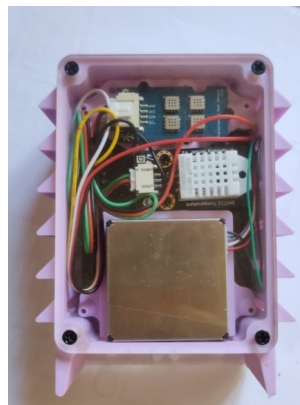


Figura 27: Estación Respira montada y funcional.

5.4 Sensorización de un aeropuerto y conectividad con Postman.

Este sería el que podríamos considerar el proyecto más grande que hicimos en nuestra estancia en el centro. Surgió gracias a la petición de Jorge, debido a la filosofía del centro.

Esta filosofía consiste en que al estar tan orientados a ser un centro en el que se aprende de forma libre al salir lo idóneo es también dejar algo de conocimiento en el centro, aportar algo.

Nos costó un par de días decidirnos sobre el tema, días en los que estuvimos probando diversos sensores y dispositivos con conectividad con Arduino (como una palanca de control, un sensor de rotación (que por desgracia no conseguimos acabar de hacer funcionar) o unos sensores de posición). Al final, recordando el proyecto del aeropuerto que hicimos en Postman (4.1) optamos por la idea de sensorizar un hipotético aeropuerto.

La idea se recibió positivamente así que nos pusimos a planear y diseñar.

Desde el principio planteábamos que el proyecto girara entorno a Arduino y los tutores consideraron que sería buena idea la de añadir conectividad con Postman, ya que al final es lo que conecta con Fiware y es en lo que está especializado el centro.

Finalmente nos dio tiempo a sensorizar dos elementos:

1º: Jorge (que nos contó que es bastante aficionado a todo lo que tiene que ver con los aviones y la organización de los mismos en el aeropuerto) nos contó que toda la gestión de si hay aviones en pista y cuales están libre y toda la información de esa índole es suministrada y gestionada de forma analógica, a través de los trabajadores que hay en el aeropuerto. Estos son los encargados de informar a los pilotos de si han de esperar porque la pista está ocupada.

Nuestra idea consistió en sensorizar el principio de la pista y (a modo de ejemplo) sensorizar una que se cruce gracias a sensores de detección de presencia. Debido a que el código para hacerlo funcionar es sencillo se podría considerar un sistema modular añadiendo las secciones de código pertinentes a los sensores que se fueran añadiendo, con objeto de tener todos los posibles cruces sensorizados.

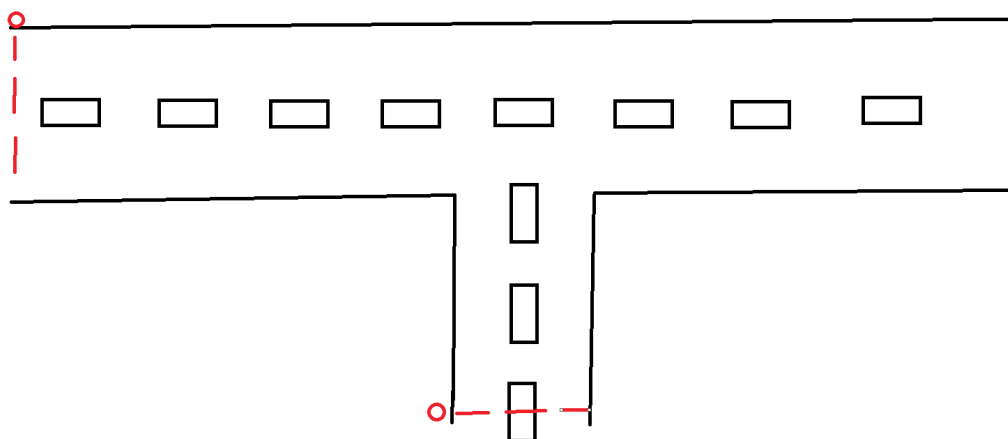


Figura 28: Esquema de la hipotética pista con los sensores.

A la hora de montarlo en la placa Arduino y con objeto de comprobar el funcionamiento colocamos un led RGB por sensor que estaría en verde si no detectaba nada y cambiaría a rojo si había algo.

La idea es que si ambos sensores detectaran presencia en una ventana determinada de tiempo (es decir, hubiera dos aviones cruzando a la vez) mandara una señal de alerta que les permitiera saber que debían detenerse o cambiar de pista. Asimismo podrían, gracias a que en Postman estaría la información de si el carril estuviera ocupado en todo momento, se podrían mantener actualizados.

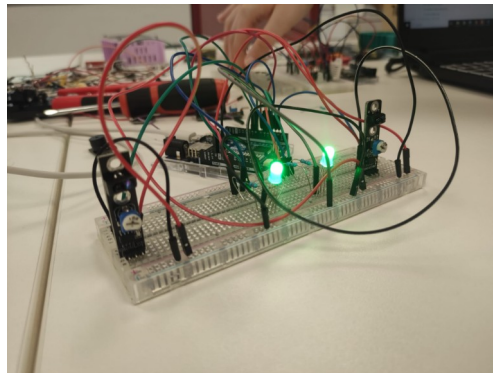


Figura 29: Montaje del circuito.

```
{
  "id": "Carril01",
  "type": "Carril",
  "Libre": {
    "type": "Boolean",
    "value": false,
    "metadata": {}
  },
  "refpistaAterrizaje001": {
    "type": "Relationship",
    "value": "pistaAterrizaje001",
    "metadata": {}
  }
},
{
  "id": "Carril02",
  "type": "Carril",
  "Libre": {
    "type": "Boolean",
    "value": false,
    "metadata": {}
  },
  "refpistaAterrizaje001": {
    "type": "Relationship",
    "value": "pistaAterrizaje001",
    "metadata": {}
  }
}
```

Figura 30: Entidades asociadas a cada carril en Postman.

(Como apunte respecto a las *figuras 29 y 30*, como se puede observar en la 29 no detecta nada, esto es debido a las luces verdes pero sin embargo vemos que en la *figura 30* ambos

carriles aparecen con la cualidad *Libre* en falso, esto es debido a que las imágenes se tomaron en momentos distintos, no de forma simultánea). Cabe destacar también que al principio el montaje era realizado con sensores del tipo avoid:

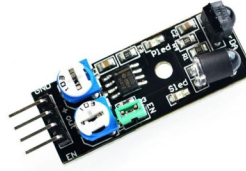


Figura 31: Sensor tipo Avoid.

Sin embargo estos sensores no paraban de dar problemas y de funcionar de forma muy poco óptima. Finalmente después de mucho ensayo y error, así como ajustar las dos ruedas que se pueden ver en la figura (que una ajusta la frecuencia a la que emite el sensor y la otra ajusta la distancia) no conseguimos hacerlos funcionar correctamente. Pero Sarah propuso cambiar este tipo de sensores por uno KY-033 (los que se ve en la imagen):



Figura 32: Sensor KY-033

Gracias a esto lo conseguimos hacer funcionar exactamente como deseábamos por lo que hay que tener que agradecer a Sarah que hiciera esa propuesta tan acertada, que nos permitió acabar esta parte del proyecto.

Ambos sensores tienen un funcionamiento similar pero, ambos emiten radiación infrarroja que si rebota contra algo en su trayecto rectilíneo volverá y acabará en el sensor que tienen incorporados lo que hará que envíen la señal pertinente.

2º: Esta segunda parte, más simple fue la implementación de un sensor DHT 22, conocido sensor de muy reducido coste que permite la detección de humedad y temperatura, valores que volveríamos a añadir a Postman para que se pudieran consultar en cualquier momento. También consideramos que era oportuno hacer un detector de niebla, parecía algo importante sobre lo que tener información en un aeropuerto, por desgracia no hay un sensor específicamente de niebla creado para Arduino.

Sin embargo si que había LDR en el centro entonces gracias al DHT y su detección de humedad, combinándolo en el código con una condición que provenga del LDR (que como ve su resistencia aumentar si se reduce la luz que recibe) en la cual si el valor de el mismo está por debajo de un umbral así como la humedad captada por el DHT está por encima de un valor concreto detecta estas dos condiciones como la presencia de niebla.

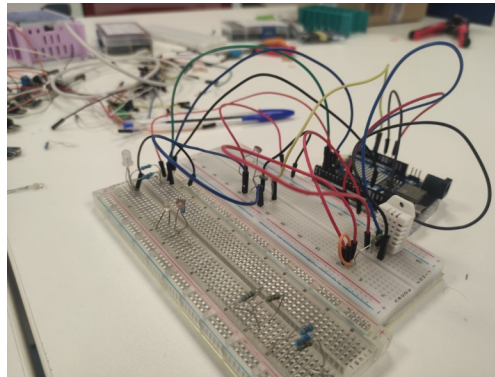


Figura 33: Montaje del DHT y LDR.

```
{
  "id": "pistaAterrizaje:pistaAterrizaje001",
  "type": "pistaAterrizaje",
  "TimeInstant": {
    "type": "DateTime",
    "value": "2025-07-11T12:01:13.547Z",
    "metadata": {}
  },
  "h": {
    "type": "Text",
    "value": 58.8,
    "metadata": {
      "TimeInstant": {
        "type": "DateTime",
        "value": "2025-07-11T12:01:13.547Z"
      }
    }
  },
  "luz": {
    "type": "Text",
    "value": 80,
    "metadata": {
      "TimeInstant": {
        "type": "DateTime",
        "value": "2025-07-08T11:53:56.751Z"
      }
    }
  },
  "temperatura": {
    "type": "Number",
    "value": 27.4,
    "metadata": {
      "TimeInstant": {
        "type": "DateTime",
        "value": "2025-07-11T12:01:13.547Z"
      }
    }
  },
  "niebla": {
    "type": "Boolean",
    "value": 0,
    "metadata": {
      "TimeInstant": {
        "type": "DateTime",
        "value": "2025-07-11T12:01:13.547Z"
      }
    }
  }
},
```

Figura 34: Datos recabados en Postman.

Con esto dimos por finalizado el proyecto. Se llegó incluso a plantear el aprender a usar el programa Grafana para mostrar los datos de forma visual pero no nos dio tiempo por desgracia.

Pese a eso el resultado lo considero correcto y estoy contento, más aun teniendo en cuenta los quebraderos de cabeza que dieron los sensores de la primera parte, que fue probablemente lo más complicado.

Asimismo nos permitió ver un caso práctico de conectividad entre sensores y Postman, lo cual es algo bastante útil.

6. Conclusiones.

Y con esto hemos terminado de repasar todo el trabajo realizado en este periodo de prácticas. Ha sido una experiencia formativa bastante enriquecedora que nos ha permitido conocer y obtener las bases para poder desarrollarnos profesionalmente en un gran número de posibilidades distintas. Ya solo será cuestión de decidir que nos interesa más y por donde querremos tirar cuando llegue el momento.

Con anterioridad mencioné en más de una ocasión que hemos usado software libre. De hecho también hablé sobre como la tecnología Fiware gira entorno a esta idea.

Me parece algo muy positivo que haya aquí (y en cualquier lugar) un lugar que impulse esta filosofía, a través de el software libre todo se vuelve más accesible y gracias a lugares como el propio centro FIWARE Space que también ofrece cursos para aprender habilidades muy valoradas. Esto permite tener más gente más formada e incluso puede permitir dar nuevas oportunidades a mucha gente.

Por último agradecer a la gente del centro pero en concreto a nuestros tutores Miguel y Jorge que nos ayudaron mucho e hicieron de nuestra estancia un tiempo muy agradable.

También agradecerle a mi compañera Sarah ya que sin ella la estancia se podría haber hecho algo más pesada como también su inestimable ayuda permitió que los proyectos que acabamos realizando llegaran a buen puerto.

Mirando atrás se trata de una experiencia positiva que abre muchas posibilidades en el futuro, posibilidades que quizás no existirían antes o al menos habrían tardado más en llegar.

7. Bibliografía.

- Arduino Modules. “Arduino Modules.info KY-032 Avoidance sensor”.
<https://arduinomodules.info/ky-032-infrared-obstacle-avoidance-sensor-module> [Última consulta: 11 julio 2025].
- Arduino Modules. “Arduino Modules.info KY-033 Line Tracking sensor”.
<https://arduinomodules.info/ky-033-line-tracking-sensor-module> [Última consulta: 11 julio 2025].
- Calidad medioambiental. “Calidad medioambiental: Respira”
<http://www.calidadmedioambiental.org> [Última consulta: 28 julio 2025].
- FIWARE Foundation. “FIWARE Space”. <https://fiware.space> [Última consulta: 15 julio 2025].
- Orange Data Mining. “Orange Data Mining Documentation”.
<https://orangedatamining.com/docs> [Última consulta: 13 junio 2025].
- W3 Schools. “W3 Schools Javascript”, “W3 Schools Python” & “W3 Schools JavaScript”
<https://www.w3schools.com> [Última consulta: 11 julio 2025].